

Software Engineering Lab 4: Vibe Coding Report

AI-Assisted Pair Programming & Feature Implementation for Frogger

Student ID: PES1UG24AM362	Repository: theDEMIGOD600/PES1UG24AM362_SE_LABs
Course: Software Engineering Lab	Tool Used: Gemini 3.8 Flash (Vibe Coding Assistant)
Base Assignment: SETAPESU26/01_frogger	Status: All 4 Tasks Completed with Separate Commits

1. Lab Overview & Objectives

The objective of this lab is to utilize modern AI pair-programming (vibe coding) tools to analyze an existing Pygame-based Frogger implementation, identify and fix existing defects, and incrementally engineer new features according to the lab specifications. Each task is developed through targeted prompts, reviewed critically, tested programmatically, and committed separately to git.

2.1 Task 1: Fix Vehicle Collision Detection

Prompt: "Inspect game/collisions.py and vehicle.py in the Frogger game. The frog is passing through moving vehicles without collisions triggering reliably. Diagnose why the collision logic fails and provide an accurate fix using Pygame's bounding box collision system."

Root Cause Analysis: In the original collisions.py, collision was calculated using `vehicle_col = int(v.x // CELL_SIZE)` and comparing column indices. Because vehicles move smoothly in float pixel increments (speed 1.5, -2.0, etc.) and have widths of 40-70 pixels spanning multiple cell boundaries, the integer floor division missed overlap on nearly all frames.

Solution: Replaced the heuristic grid column check with `pygame.Rect.colliderect()` checking intersection between `frog.get_rect(CELL_SIZE)` and `vehicle.get_rect(CELL_SIZE)`.

Git Verification: Commit: Task 1: Fix vehicle collision detection using bounding box intersection

2.2 Task 2: Implement Lives and Respawn System

Prompt: "Implement a 3-life system for Frogger with proper respawn behaviour. When a collision occurs, the frog should respawn at its starting position and lose 1 life. When all 3 lives are exhausted, trigger a Game Over state, lock frog movement, and display a game-over banner. Allow pressing 'R' to restart."

Architecture Design: Extended GameEngine with `max_lives = 3`, `lives = 3`, and `game_over = False`. Integrated collision handling into `update()` to decrement lives and reset frog coordinates. Enhanced `renderer.py` with formatted banner rendering and HUD display.

Solution: Added `restart()` method, state guards in `handle_keydown()` to prevent input after Game Over, and displayed live life count in the game HUD.

Git Verification: Commit: Task 2: Implement 3-life system, frog respawn, and game over state

2.3 Task 3: Implement Goal Completion and Score Tracking

Prompt: "Implement goal detection when the frog reaches row 0 (the goal zone). Add score tracking (+100 points on reaching the goal), display the score in the HUD, and trigger an appropriate win state with a banner. Pressing 'R' should reset the game and score."

Architecture Design: Added score = 0 and game_won = False to GameEngine. In update(), detected when frog.row == GOAL_ROW (row 0), awarding points and setting the win state. Input handling prevents frog movement during win state until restarted with 'R'.

Solution: Updated HUD rendering to show Lives: X Score: Y and rendered victory banner YOU WON! Score: ... - Press R to Restart.

Git Verification: Commit: Task 3: Implement goal completion detection, score tracking, and win state

2.4 Task 4: Add 30-Second Countdown Timer & Timeout Handling

Prompt: "Add a 30-second countdown timer for each attempt in Frogger. The timer should display in the HUD and count down in real-time. If the timer reaches 0, the frog must lose a life, respawn at start, and reset the attempt timer to 30s. If lives reach 0, trigger Game Over. If the frog wins, award a time bonus based on seconds remaining."

Architecture Design: Added attempt_time_limit = 30.0 and time_remaining. In main.py, calculated delta time dt = clock.tick(60) / 1000.0 and passed to engine.update(dt). Implemented centralized _handle_life_lost() to manage life deduction and attempt timer resets for both vehicle collisions and timeouts.

Solution: Displayed countdown in HUD with warning color (red) under 5 seconds. Awarded time bonus int(round(time_remaining)) * 10 on goal completion.

Git Verification: Commit: Task 4: Add 30-second countdown timer per attempt and timeout handling

3. Verification and Testing

All features were verified through automated unit test scripts verifying collision detection under multiple offsets, life decrement & game over triggers, goal reaching & bonus calculation, and attempt timer countdown & timeout respawns. The Pygame window runs stably at 60 FPS without memory leaks or dropped frames.

4. Summary of Deliverables in Lab-4 Folder

- **frogger/:** Fully updated source code with all 4 tasks implemented.
- **videos/:** Directory containing instructions and placeholders for 10s before & after demo videos.
- **Lab_4_Chat_History.pdf / .docx:** Exported documentation of prompts and vibe coding interactions.